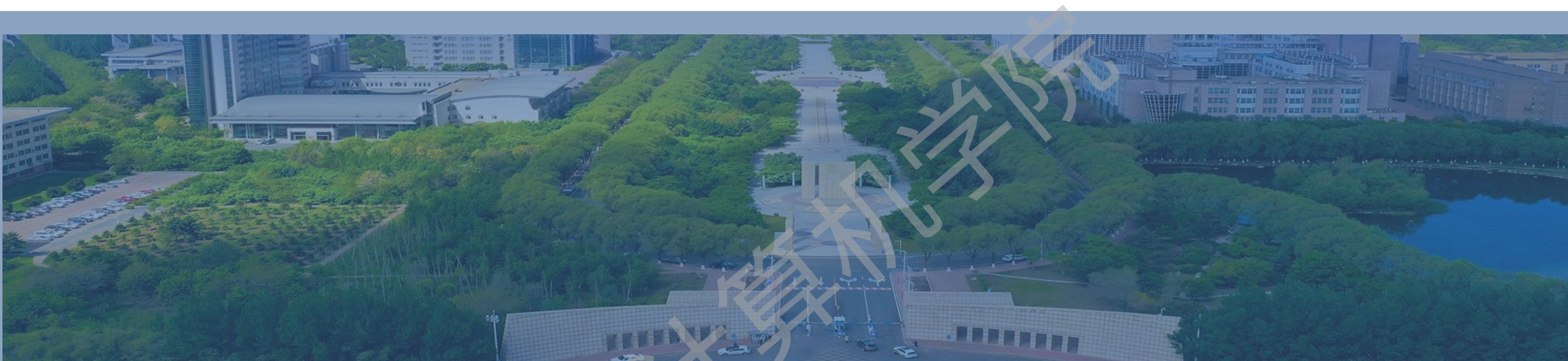




第二章 导引与基本数据结构



目录

- 2.1 算法
- 2.2 分析算法
- 2.3 用SPARKS语言写算法
- 2.4 基本数据结构 (略)





2.1 算法

- 什么是算法
- 算法的五个重要特性
- 计算过程与算法的区别
- 问题的求解过程
- 算法学习的基本内容



什么是算法

- 算法是解决一确定类问题的任意一种特殊的方法。
 - 数值计算方法：求解数值问题，如插值计算、数值积分等。
 - 非数值计算方法：求解非数值问题，主要进行判断比较。
- 算法的非形式描述：算法就是一组有穷的规则，它规定了解决某一特定类型问题的一系列运算。

Knuth给出的概念





算法(Algorithm)的五个重要特性

- 满足如下五条特性的一组规则才能被称为算法
 - 确定性 (Definiteness)
 - 能行性 (Effectiveness)
 - 输入 (Input)
 - 输出(Output)
 - 有穷性 (Finiteness)



算法(Algorithm)的五个重要特性

- **确定性**：每一种运算必须要有确切定义，无二义性。
- **能行性**：运算都是基本运算，原理上能用纸和笔在有限时间完成。
- **输入**：有0个或多个输入。
 - 在算法开始之前，从特定的对象集合中取值。
- **输出**：一个或多个输出。
 - 这些输出和输入有特定关系。
- **有穷性**：在执行了有穷步运算后终止。

反例：5/0;
6或7与x相加

非常有限，要在现代计算机上有效才行

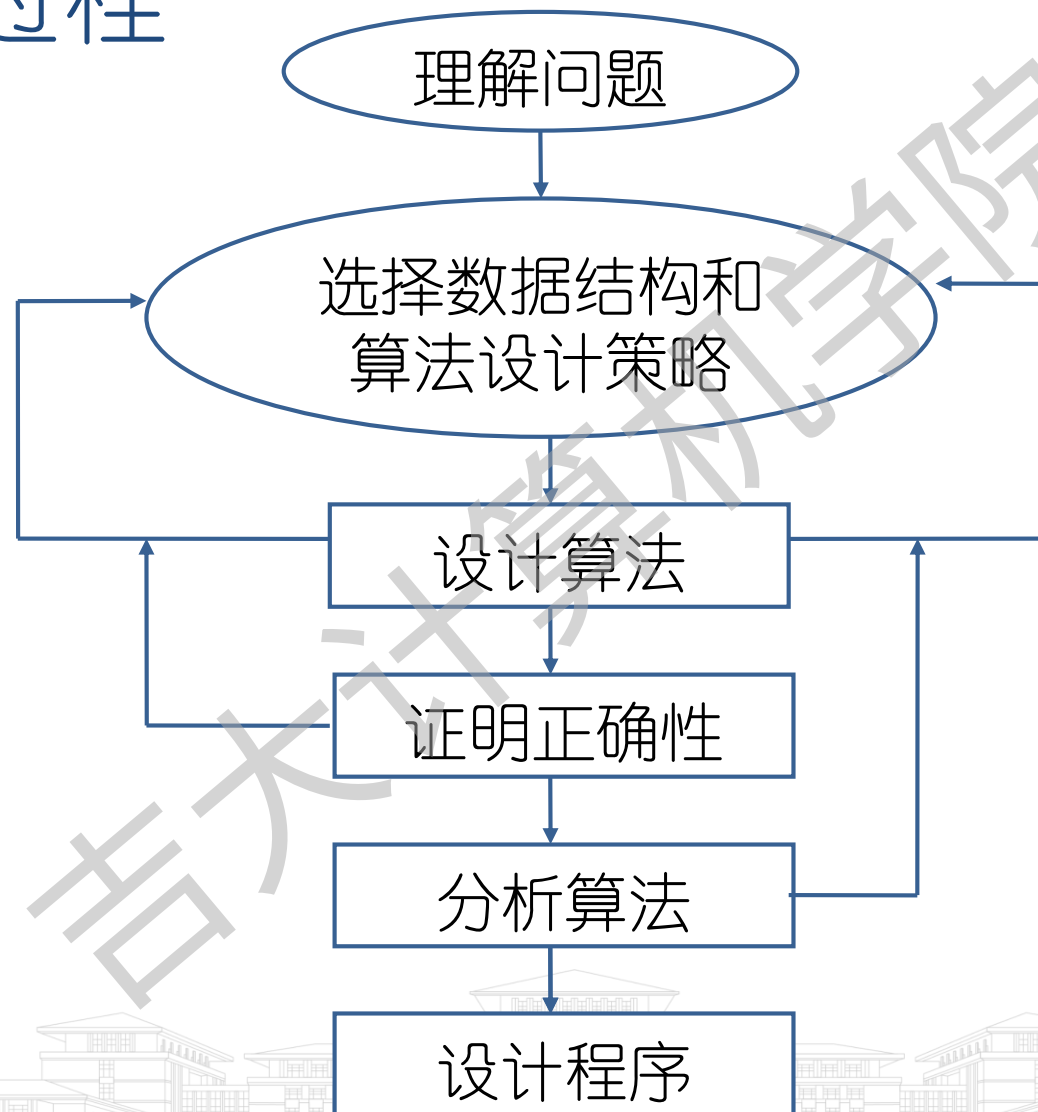


计算过程与算法的区别

- 计算过程可以不满足算法的性质(5)有穷性。
- 例如操作系统，当没有任务时，操作系统并不终止，而是处于等待状态，直到有新的任务启动，因而不是一个算法。
- 程序 = 算法 + 数据结构 (By Niklaus Wirth, 1984图灵奖得主)



问题的求解过程





算法学习的基本内容

- 如何设计算法
- 如何表示算法
- 如何确认算法
- 如何分析算法
- 如何测试程序



如何设计算法

- 面对具体问题，运用一些基本设计策略，规划算法
 - 被实践证明是有用的
 - 计算机科学、运筹学、电器工程等领域
 - 设计出很多精致有效的好算法
- 掌握这些策略，设计出更多的好算法





如何表示算法

- 设计的算法要用恰当的方式地表示出来
- 采用结构程序设计的方式
- SPARKS程序设计语言——简单明了





如何确认算法

- 算法确认(**algorithm validation**)——证明该算法对所有可能的合法输入，都能给出正确答案
- 算法确认的目的——确保该算法能正确无误地工作
 - 穷举法
 - 推理——数学归纳法、反证法
 - 构造性证明
 - 测试



如何分析算法

- 分析执行一个算法时，
 - 占用多少CPU时间完成运算；
 - 占用多少存储器的存储空间，存放程序和数据。
- 既对一个算法需要多少计算时间和存储空间，做定量分析。



测试程序

- 调试——调试只能指出有错误，而不能指出它们不存在错误。
 - 源于程序正确性的证明还没有取得突破性进展。
- 时空分布图
 - 用各种给定数据执行调试后的程序
 - 测定计算时间和空间
 - 印证之前的分析是否正确
 - 指出实现最优化的有效逻辑位置



2.2 分析算法

- 算法分析目的
- 算法分析的准备工作的
- 计算时间的渐进表示
- 一些证明方法
- 作时空性能分布图



算法分析的目的

- 算法分析(analysis of algorithms)是对一个算法需要多少**计算时间**和**存储空间**作定量的分析。
 - 确定算法在什么样的环境下能够有效地运行。
- 分析在**最好**、**最坏**和**平均**情况下的执行情况。
 - 对同一问题不同算法的有效性作出比较。



时间复杂性的形式化定义

- 算法的时间复杂性 $T(n)$;
- 算法的空间复杂性 $S(n)$;
- 其中 n 是问题的规模。

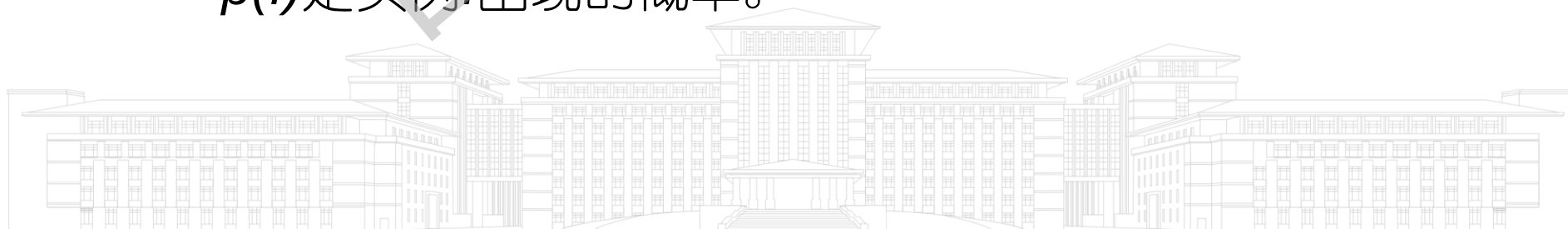
最坏情况下: $T_{\max}(n) = \max\{ T(I) \mid \text{size}(I)=n \}$

最好情况下: $T_{\min}(n) = \min\{ T(I) \mid \text{size}(I)=n \}$

平均情况下: $T_{\text{avg}}(n) = \sum_{\text{size}(I)=n} p(I)T(I)$

其中 I 是问题的规模为 n 的实例,

$p(I)$ 是实例 I 出现的概率。



算法运行假定的计算机类型

- 要求独立于具体的软硬件环境单纯分析算法。
- 假定使用一台通用计算机
 - 顺序处理每条指令；
 - 存储容量足够大；
 - 存取时间恒定。



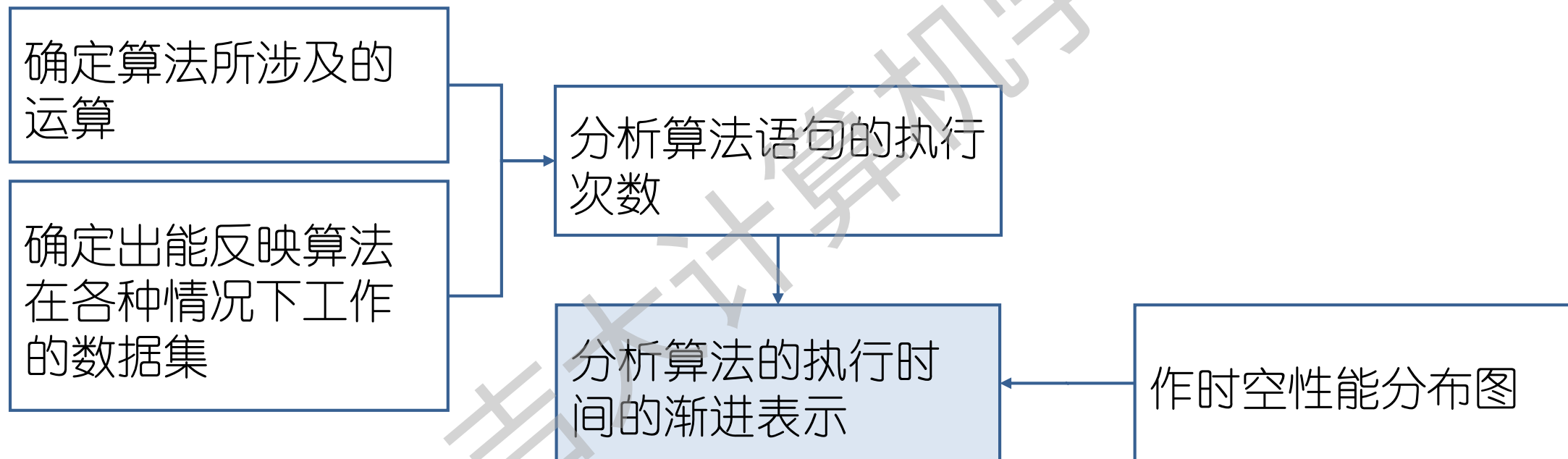
算法分析过程

全面分析的两个阶段

准备工作

事前分析

事后测试



准备工作

- 首先确定使用哪些**运算**以及执行这些运算所用的时间。
 - 基本运算
 - 由一些更基本的任意长序列的运算所组成的复杂运算
- 其次是要确定出能反映算法在各种情况下工作的数据集。
 - 编造出能产生**最好、最坏和有代表性**情况的数据配置
 - 通过使用这些数据来运行算法，以更好地了解算法的性能

算法分析最重要和最富于创造性的工作。



基本运算

- 时间囿界于常数的运算
 - 加、减、乘、除整数算术运算
 - 浮点算术、字符比较、对变量赋值、过程调用等
- 每种运算所用时间虽然不同，但都只花费一个固定量的时间



复杂运算

- 由一些更基本的任意长序列的运算组成，如：两个字符串的比较运算
 - 一系列字符比较指令
 - 一个字符的比较时间——囿界于常数
 - 字符串比较的时间总量则取决于字符串的长度



全面分析算法的两个阶段

- 事前分析(a priori analysis)
 - 确定每条语句的执行次数
 - 求出该算法的一个时间限界函数(关于问题规模 n 的函数)
- 事后测试(a posteriori testing)
 - 作时空性能分布图



算法的执行时间

- 同一条语句在一个算法中的执行次数 (frequency count)称为频率计数
 - 由算法直接确定，与所用的机器无关，且独立于程序设计语言。
- 语句的时间总量=频率计数 × 执行一次该语句所需要的时间
 - 语句本质上是由运算组成的
 - 语句的执行时间依赖于机器、编程语言、编译程序
- 算法的执行时间就是构成算法的所有语句的执行时间总量之和





例：计算语句 $x \leftarrow z + y$ 在下面三个程序段中的频率计数

程序段1

```
begin
 $x \leftarrow z + y$ 
end
```

1次

程序段2

```
begin
for  $i \leftarrow 1$  to  $n$  do
     $x \leftarrow z + y$ 
end
```

n次

程序段3

```
begin
for  $i \leftarrow 1$  to  $n$  do
    for  $j \leftarrow 1$  to  $n$  do
         $x \leftarrow z + y$ 
    end
end
```

n^2 次

- 语句的数量级是指执行它的频率
- 算法的数量级是指算法的所有语句的执行频率之和

确定一个算法的数量级十分重要，因为它在本质上反映了算法所需的计算时间。

计算时间的基本特性

- 描述算法数量级的多项表达式 ×
- 最高次项 ×
- 最高次项的系数 ×
- 最高次项的次数 ✓

准确分析出算法数量级的多项式表达式是很困难的，因此我们对事前分析的计算时间进行渐进表示。



总结



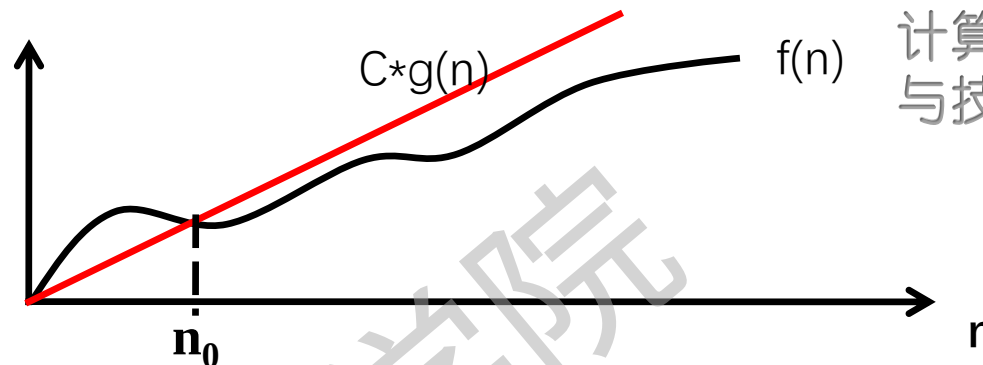


计算时间的渐进表示

- 定义2.1: $f(n) = O(g(n))$
- 定义2.2: $f(n) = \Omega(g(n))$
- 定义2.3: $f(n) = \Theta(g(n))$
- 定理2.1
- n 表示问题规模,
 - 输入或输出量;
 - 两者之和;
 - 其中之一的一种测度。
- $f(n)$ 表示算法的计算时间。
- $g(n)$ 是在事前分析中确定的某个形式简单的函数。

$f(n)$ 与机器和语言有关, 而 $g(n)$ 是独立于机器和语言的。

定义2.1



如果存在两个正常数 c 和 n_0 ，对于所有的 $n \geq n_0$ ，有 $|f(n)| \leq c|g(n)|$ ，则记作： $f(n) = O(g(n))$ 。

- 当 n 充分大时， $f(n)$ 有上界，一个常数倍的 $g(n)$ 是 $f(n)$ 的一个上界， $f(n)$ 的数量级就是 $g(n)$ 。
- $f(n)$ 的阶不高于 $g(n)$ 的阶。
- 在确定 $f(n)$ 的数量级时，总是试图求出最小的 $g(n)$ 。
- 有关证明中，找出 c 和 n_0 是关键。

判断 $f(n) = O(g(n))$?

基于定义证明你的判断

• $f(n) = 3n$, $g(n) = n$ 证明: $\exists c=3, n_0=1, \forall n \geq n_0$, 均有 $3n \leq cn$ 。证毕。

• $f(n) = n+1024$, $g(n) = 1025n$

• $f(n) = 2n^2+11n-10$, $g(n) = 3n^2$

• $f(n) = n^2$, $g(n) = n^3$

• $f(n) = n^3$, $g(n) = n^2$

证明:

假设 $\exists$ 正常数 c 和 n_0 , 使得 $n \geq n_0$ 时, $n^3 \leq cn^2$ 成立。

构造 $n_1 = \max\{n_0, c\} + 1$, 显然有 $n_1 > n_0$ 且 $n_1 > c$, 显然 $n_1^3 > cn_1^2$, 与假设矛盾, 故 $n^3 \neq O(n^2)$ 。证毕。



O 性质

对于非负的 $f(n)$ 和 $g(n)$ ，根据定义2.1，有如下性质：

1. $O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$;
2. $O(f(n)) + O(g(n)) = O(f(n) + g(n))$;
3. $O(f(n))O(g(n)) = O(f(n)g(n))$;
4. 如果 $g(n) = O(f(n))$, 则 $O(f(n)) + O(g(n)) = O(f(n))$;
5. $O(cf(n)) = O(f(n))$, 其中 c 是一个正的常数;
6. $f(n) = O(f(n))$ 。



证明 $O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\})$

不妨设 $p(n) = \max\{f(n), g(n)\}$

设 $f'(n) = O(f(n))$, 则存在 c_1, n_1 , 当 $n \geq n_1$ 时, $f'(n) \leq c_1 \times f(n)$

设 $g'(n) = O(g(n))$, 则存在 c_2, n_2 , 当 $n \geq n_2$ 时, $g'(n) \leq c_2 \times g(n)$

则 $O(f(n)) + O(g(n)) = f'(n) + g'(n)$

当 $n \geq \max\{n_1, n_2\}$ 时,

$f'(n) + g'(n) \leq c_1 \times f(n) + c_2 \times g(n) \leq c_1 \times p(n) + c_2 \times p(n) = (c_1 + c_2) \times p(n)$

即存在 $c_3 = c_1 + c_2$, $n_3 = \max\{n_1, n_2\}$, 当 $n \geq n_3$ 时, $f'(n) + g'(n) \leq c_3 \times p(n)$

故 $O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\})$





定理2.1

若 $A(n)=a_m n^m + \dots + a_1 n + a_0$ 是一个 m 次多项式，则 $A(n)=O(n^m)$ 。

证明：取 $n_0=1$ ，当 $n \geq n_0$ 时

由 $A(n)$ 的定义和不等式关系 $|A+B| \leq |A|+|B|$ 有

$$\begin{aligned} |A(n)| &= |a_m n^m + \dots + a_1 n + a_0| \leq |a_m| n^m + \dots + |a_1| n + |a_0| \\ &= (|a_m| + |a_{m-1}|/n + \dots + |a_0|/n^m) n^m \\ &\leq (|a_m| + |a_{m-1}| + \dots + |a_0|) n^m \end{aligned}$$

取 $c = |a_m| + |a_{m-1}| + \dots + |a_0|$ ，有 $|A(n)| \leq c n^m$

即： $A(n)=O(n^m)$ ，定理得证。

定理2.1：若 $A(n)=a_m n^m + \dots + a_1 n + a_0$ 是一个 m 次多项式，则 $A(n)=O(n^m)$ 。

- 定理2.1表明，变量 n 的最高阶数为 m 的任一多项式，与 n^m 同阶。因此一个计算时间为 m 阶多项式的算法，其时间都可以用 $O(n^m)$ 来表示。
- 若一个算法有数量级为 $c_1 n^{m_1}$ ， $c_2 n^{m_2}$ ，... $c_k n^{m_k}$ 的 k 个语句，则算法的数量级及计算时间就是

$$c_1 n^{m_1} + c_2 n^{m_2} + \dots + c_k n^{m_k} = O(n^m), \quad m = \max\{m_i | 1 \leq i \leq k\}$$



数量级对算法有效性的影响

- 从计算时间上算法可以分为两类：
 - 多项式时间算法(**polynomial time algorithm**): 计算时间用多项式函数限界的算法

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3)$$

- 指数时间算法(**exponential time algorithm**): 计算时间用指数函数限界的算法

$$O(2^n) < O(n!) < O(n^n)$$



不同时间复杂性函数的对比

T(n) 单位	logn	n	nlogn	n^2	n^3	n^5	2^n	3^n	$n!$
n=10	3.3	10	33	100	1 毫秒	0.1 秒	1 毫秒	59 毫秒	3.6 秒
n=40	5.3	40	213	1600	64 毫秒	1.3 分	12.7 天	3855 世纪	10^3 世纪
n=60	5.9	60	354	3600	216 毫秒	7.6 分	366 世纪	1.3×10^{13} 世纪	10^{66} 世纪

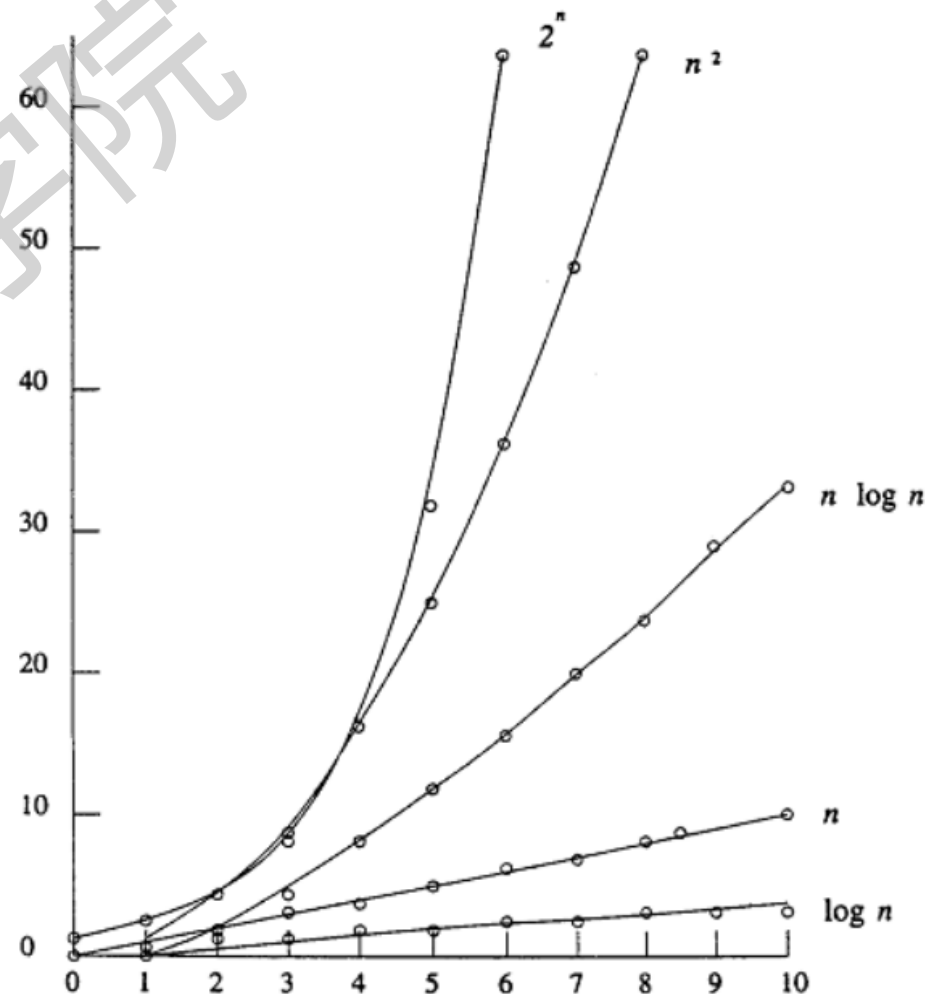


图2-1 时间复杂度函数曲线 blog.csdn.net/qq_41856733

对数函数的优越属性

n	$\log_2 n$
10	3.3
10^2	6.6
10^3	10
10^4	13
10^5	17
10^6	20

- 对数函数的增长如此之慢，几乎可以认为：如果一个算法具有对数级的基本操作次数，该程序对任何实际规模的输入几乎都会在瞬间完成。



定义2.2

如果存在两个正常数 c 和 n_0 ，对于所有的 $n \geq n_0$ ，有 $|f(n)| \geq c|g(n)|$ ，则记作： $f(n) = \Omega(g(n))$ 。

- 当 n 充分大时， $f(n)$ 有下界，一个常数倍的 $g(n)$ 是 $f(n)$ 的一个下界。
- $f(n)$ 的阶不低于 $g(n)$ 的阶。
- 在确定 $f(n)$ 的下界时，总是试图求出最大的 $g(n)$ 。
- 有关证明中，找出 c 和 n_0 是关键。



Ω 性质

对于非负的 $f(n)$ 和 $g(n)$ ，根据定义2.2，有如下性质：

1. $\Omega(f(n)) + \Omega(g(n)) = \Omega(\min(f(n), g(n)))$;
2. $\Omega(f(n)) + \Omega(g(n)) = \Omega(f(n) + g(n))$;
3. $\Omega(f(n)) \Omega(g(n)) = \Omega(f(n) g(n))$;
4. 如果 $g(n) = \Omega(f(n))$ ，则 $\Omega(f(n)) + \Omega(g(n)) = \Omega(f(n))$;
5. $\Omega(cf(n)) = \Omega(f(n))$ ，其中 c 是一个正的常数；
6. $f(n) = \Omega(f(n))$ 。



定义2.3

如果存在正常数 c_1, c_2 和 n_0 , 对于所有的 $n \geq n_0$, 有 $c_1|g(n)| \leq |f(n)| \leq c_2|g(n)|$, 则记作: $f(n) = \Theta(g(n))$ 。

- 一个算法的 $f(n) = \Theta(g(n))$ 意味着该算法在最好和最坏情况下的计算时间就一个常数因子范围内而言是相同的。



渐近表示函数的若干性质

• 传递性

- $f(n) = \Theta(g(n)), g(n) = \Theta(h(n)) \Rightarrow f(n) = \Theta(h(n));$
- $f(n) = O(g(n)), g(n) = O(h(n)) \Rightarrow f(n) = O(h(n));$
- $f(n) = \Omega(g(n)), g(n) = \Omega(h(n)) \Rightarrow f(n) = \Omega(h(n));$

• 反身性

- $f(n) = \Theta(f(n));$
- $f(n) = O(f(n));$
- $f(n) = \Omega(f(n)).$



常用的整数求和公式

$$\sum_{1 \leq i \leq n} 1 = \Theta(n)$$

$$\sum_{1 \leq i \leq n} i = n(n+1)/2 = \Theta(n^2)$$

$$\sum_{1 \leq i \leq n} i^2 = n(n+1)(2n+1)/6 = \Theta(n^3)$$

通式: $\sum_{1 \leq i \leq n} i^k = \frac{n^{k+1}}{k+1} + \frac{n^k}{2} + \text{低次项} = \Theta(n^{k+1})$



一些数学证明方法

- 直接证明: $P \rightarrow Q$
- 间接证明:
 - 反证法
 - 举反例
- 数学归纳法:
 - 初始归纳: $i=1$ 结论成立;
 - 归纳假设: 若 $i=n-1$ 时成立;
 - 归纳证明: 证明 $i=n$ 时成立。



作时空性能分布图

- 事后测试是在对算法进行设计、确认、事前分析和调试之后要做的工作，
与所用计算机密切相关。
- 以时间分布图为例：
 - 为提高算法时间的准确测量，减少时钟不准确造成的噪声，可以增大规模或重复执行
 - 分布图的做法 -> 数据集与时间复杂度有关





2.3 用SPARKS语言写算法

- 基本数据类型和变量命名规则
- 运算符
- 条件语句和循环语句
- 跳转语句
- 算法定义
- 算法的同质异相



基本数据类型和变量命名规则

- 基本数据类型：

整型(integer), 实型(real), 布尔型(boolean), 字符型(char)

布尔值: true, false

- 变量命名规则：

以字母开头, 不允许使用特殊字符, 不允许与保留字重复

例: integer x, y; char ch;

- 数组：任意整数下界和上界的多维数组：integer $A(l_1:u_1, \dots, l_n:u_n)$

例: integer A(1:10); real B(3:6, 1:4);



运算符

- 赋值语句：变量 $\leftarrow$ 表达式
- 逻辑运算符：and , or , not
- 关系运算符：< , $\leq$, = , $\neq$, > , $\geq$



条件语句和循环语句

• if语句

```
if 条件 then S1
endif
```

```
if 条件 then S1
else S2
endif
```

• case语句

```
case
: 条件 1: S1
...
: 条件 n: Sn
: else : Sn+1
endcase
```

• while语句

```
while 条件 do
S
```

repeat //条件为假，循环结束

• loop语句

loop//条件为真，循环结束

```
S
```

until 条件 repeat

• for语句

```
for 变量←初值 to 终值 by 变量增值 do
```

```
S
```

repeat

跳转语句

- goto 标号
- exit
 - 转到含有**exit**的最内层循环语句后面的第一条语句
- cycle
 - 结束本次循环
- return (<表达式>)



算法定义

- SPARKS语言的输入、输出：
 - read (参数表) ; print (参数表) ;

- SPARKS语言的函数(过程)

procedure **NAME**(<参数列表>)

<说明部分>

S

return(<表达式>)

end NAME

函数名通常用大写字母

说明参数的数据类型和函数中使用的变量

parameters 形式参数

global 全局变量

local 局部变量



算法的同质异相

```

procedure MAX(A, n, j)
  parameters real A(1:n)
  parameters integer n, j
  global real xmax
  local integer i
xmax ← A(1)
for i ← 2 to n do
  if A(i) > xmax then
    xmax ← A(i); j ← i
  endif
repeat
return xmax
end MAX
  
```

```

procedure MAX(A, n, j)
  global xmax //A(1:n)
  int i, n, j
xmax ← A(1)
for i ← 2 to n do
  if A(i) > xmax then
    xmax ← A(i); j ← i
  endif
repeat
return xmax
end MAX
  
```



课程之外

- 授人以鱼不如授人以渔
- 开阔思路，训练思维
- 方法永远拥有理性的特点





本章结束

